

EMAIL SPAM DETECTION

Machine Learning Project Report

Logistic Regression + TF-IDF

Project	Email Spam Detection
Algorithm	Logistic Regression
Dataset	us_email_dataset_10000_rows.csv
Dataset Size	10,000 rows x 8 columns
Author	Reem Fayyaz

This report presents the design, implementation, evaluation, and deployment workflow of the Email Spam Detection project.

1. Executive Summary

This project develops a binary email classification system that predicts whether an email is Spam or Ham (legitimate). The system combines each email subject and body, converts the text into TF-IDF features, and trains a Logistic Regression classifier.

The project uses a balanced dataset of 10,000 emails, an 80/20 train-test split, and achieves a test accuracy of 100.00% on 2,000 test samples.

2. Problem Statement

Unwanted or malicious emails can reduce productivity and expose users to phishing or fraudulent content. The objective is to create a supervised machine-learning classifier that uses email text to distinguish Spam from Ham.

3. Dataset Overview

Field	Description
email_id	Unique email identifier
sender_name	Name of sender
sender_email	Sender email address
recipient_email	Recipient email address
subject	Email subject
email_body	Email message body
category	Email category such as Phishing or Business
label	Target class: Spam or Ham

The dataset contains no missing values in the eight original columns. After label conversion, the class distribution is balanced: 5,000 Spam samples and 5,000 Ham samples.

4. Data Preparation

The project removes duplicate records, fills missing values in subject and email_body with empty strings, and creates a combined text feature by joining those two fields. Target labels are converted to numeric values: Ham = 0 and Spam = 1.

5. Feature Engineering with TF-IDF

Email text cannot be passed directly to Logistic Regression, so the project uses TfidfVectorizer with English stop-word removal and up to 5,000 text features. TF-IDF increases the importance of informative terms while reducing the influence of very common terms.

6. Model Configuration

Parameter	Value
Algorithm	Logistic Regression
C	5

Parameter	Value
Solver	liblinear
Maximum iterations	3000
Random state	42
Training split	80%
Testing split	20%

7. Evaluation Results

Metric	Ham (0)	Spam (1)	Overall
Precision	1.00	1.00	-
Recall	1.00	1.00	-
F1-score	1.00	1.00	-
Support	993	1007	2000
Accuracy	-	-	100.00%

Confusion Matrix: $\begin{bmatrix} 993 & 0 \\ 0 & 1007 \end{bmatrix}$. All 993 Ham test samples and all 1,007 Spam test samples were classified correctly.

Validation note: A 100% test score is unusually strong for real-world email classification. Independent validation on new external email data is recommended to check generalization and rule out dataset simplicity, duplication patterns, or other forms of leakage.

8. Saved Model Artifacts

The project saves two files using Joblib: **spam_model.pkl** for the trained Logistic Regression classifier and **tfidf_vectorizer.pkl** for the fitted TF-IDF vectorizer. Both files are required for deployment because new email text must be transformed using the same vocabulary learned during training.

9. Prediction Workflow

For a new email, the application combines the entered subject and body, transforms the resulting text with the saved TF-IDF vectorizer, and calls the saved model to produce a class prediction. If probability estimates are available, it also displays Ham and Spam confidence percentages.

Step	Operation
1	User enters subject and/or email body
2	Subject and body are combined into one text string
3	Saved TF-IDF vectorizer transforms the text
4	Logistic Regression predicts class 0 or 1

Step	Operation
5	Application displays Ham/Spam result and confidence

10. Streamlit Deployment

A companion Streamlit application loads the two saved PKL files and provides a browser-based interface for email screening. For deployment, keep **app.py**, **requirements.txt**, **spam_model.pkl**, and **tfidf_vectorizer.pkl** in the project repository.

11. Conclusion

This project demonstrates a complete text-classification workflow: cleaning email data, combining text fields, extracting TF-IDF features, training Logistic Regression, evaluating the classifier, saving reusable artifacts, and predicting new user input. The evaluation is perfect on the current test split; the next practical step is evaluation on independent real-world emails before production use.